UTS Sistem Terdistribusi dan Parallel A

"Pub-Sub Log Aggregator dengan Idempotent Consumer dan Deduplication"

LAPORAN UTS INDIVIDU

Disusun oleh:
Nurfauzan Gymnastiar
11231073

PROGRAM STUDI INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN
2026

DAFTAR ISI

DAFTAR ISI i
BAB I PENDAHULUAN 2
Gambar 1.1 Sistem Distribusi Paralel 2
BAB II PENJELASAN RINGKAS SISTEM DAN ARSITEKTUR 3
Gambar 1.2 Tampilan VS CODE Struktur kode pada tugas UTS 3
BAB III Karakteristik Sistem Terdistribusi dan Trade-off pada Pub-Sub Log Aggregator 4
BAB IV Arsitektur Client-Server vs Publish-Subscribe untuk Log Aggregator 4
BAB V At-Least-Once vs Exactly-Once Delivery Semantics 5
BAB VI Skema Penamaan untuk Topic dan Event ID 5
BAB VII Failure Modes dan Strategi Mitigasi 6
Table 1.1 Table failure mode 6
BAB VIII Eventual Consistency dan Idempotency dalam Aggregator 7
BAB XI Metrik Evaluasi Sistem (Bab 1–7) Buku 7
Tujuan 1.2 Table Evaluasi Sistem dari buku. 7
BAB XII ALUR EKSEKUSI SISTEM 8
DAFTAR PUSTAKA 11

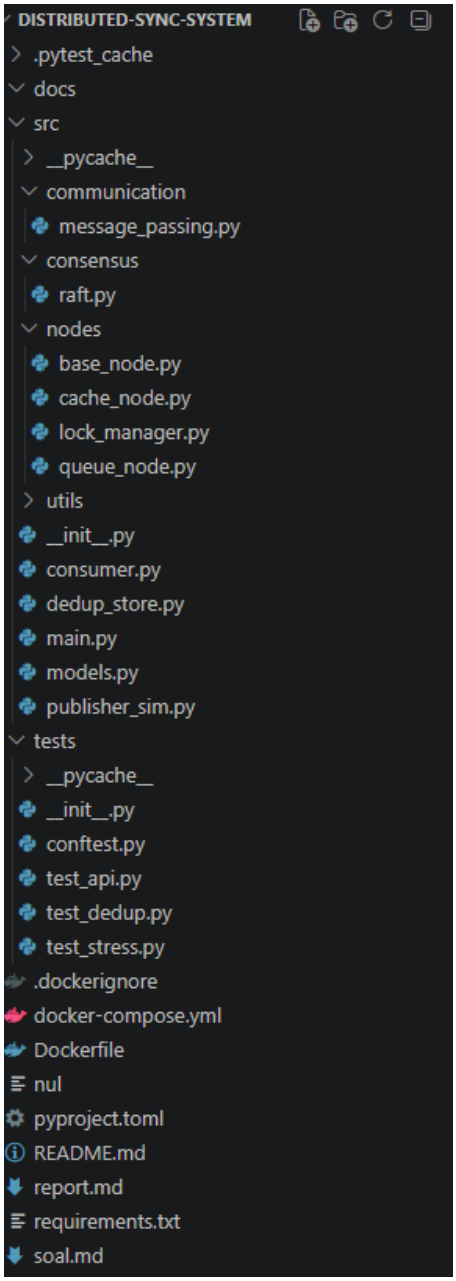
BAB I
PENDAHULUAN

Gambar 1.1 Sistem Distribusi Paralel

Sistem terdistribusi merupakan sistem yang terdiri dari beberapa komponen yang saling berinteraksi melalui jaringan. Salah satu tantangan utama dalam sistem ini adalah duplicate delivery, yaitu kondisi ketika suatu event dikirim lebih dari satu kali akibat retry atau kegagalan jaringan. Hal ini dapat menyebabkan inkonsistensi data jika tidak ditangani dengan baik.

Untuk mengatasi permasalahan tersebut, sistem ini menggunakan pendekatan idempotent consumer dan deduplication. Dengan pendekatan ini, setiap event yang memiliki identitas yang sama hanya akan diproses satu kali, meskipun dikirim berulang kali.

BAB II
PENJELASAN RINGKAS SISTEM DAN ARSITEKTUR



Gambar 1.2 Tampilan VS CODE Struktur kode pada tugas UTS

Sistem ini dibangun menggunakan arsitektur publish-subscribe. Publisher mengirimkan event ke aggregator melalui REST API. Event yang diterima akan masuk ke dalam antrian asynchronous (asyncio.Queue) sebelum diproses oleh consumer.

Consumer kemudian memeriksa apakah event tersebut sudah pernah diproses dengan melihat kombinasi (topic, event_id) pada database SQLite. Jika belum, event akan diproses dan disimpan. Jika sudah ada, event akan diabaikan sebagai duplikat.

BAB III
Karakteristik Sistem Terdistribusi dan Trade-off pada Pub-Sub Log Aggregator

Tanenbaum dan Van Steen (2023) mendefinisikan sistem terdistribusi sebagai sekumpulan komponen komputasi independen yang tampak bagi pengguna sebagai satu sistem terpadu.
Empat karakteristik utama yang relevan dengan Pub-Sub log aggregator adalah:

(1) *concurrency* — komponen publisher dan consumer berjalan secara bersamaan;
(2) *no global clock* — tidak ada jam tunggal yang disepakati, sehingga ordering berdasarkan timestamp fisik rentan terhadap *clock skew*;
(3) *independent failures* — publisher dapat gagal tanpa mempengaruhi consumer, dan sebaliknya;
(4) *scalability* — sistem harus tetap performan seiring bertambahnya jumlah event dan topic.

Trade-off utama pada desain Pub-Sub log aggregator adalah antara *consistency* dan *availability* (Brewer's CAP theorem). Jika aggregator memilih *high availability* (tetap menerima event meski consumer lag), maka *consistency* (semua event pasti diproses tepat sekali) menjadi lebih sulit dijamin. Pilihan *exactly-once* semantics meningkatkan kompleksitas dan latency, sementara *at-least-once* lebih mudah diimplementasikan namun mengharuskan consumer bersifat idempotent. Dalam sistem ini, dipilih *at-least-once* dengan idempotent consumer berbasis SQLite, sehingga trade-off bergeser ke arah *availability* dan *simplicity* tanpa mengorbankan *correctness* (Tanenbaum & Van Steen, 2023, Bab 1).

BAB IV
Arsitektur Client-Server vs Publish-Subscribe untuk Log Aggregator

Tanenbaum dan Van Steen (2023, Bab 2) membedakan dua pola arsitektur utama: *client-server* dan *event-driven/publish-subscribe*. Pada arsitektur client-server, setiap producer log harus mengetahui alamat (host:port) aggregator secara eksplisit; komunikasi bersifat *synchronous* dan *point-to-point*. Ini menciptakan *tight coupling*: jika aggregator down, semua producer terdampak langsung.

Sebaliknya, arsitektur Pub-Sub memisahkan producer (*publisher*) dari consumer (*subscriber*) melalui perantara (*broker* atau *event bus*). Publisher hanya peduli pada *topic*, bukan siapa yang mendengarkan. Consumer mendaftar ke topic yang relevan dan menerima event secara *asynchronous*. Hal ini memberikan *loose coupling*, *scalability*, dan *fault tolerance* yang lebih baik.

Kapan memilih Pub-Sub?

Pub-Sub lebih tepat ketika:
(a) terdapat banyak producer yang menghasilkan event secara independen;
(b) consumer dapat bertambah atau berkurang tanpa mengubah producer;
(c) sistem harus toleran terhadap *consumer lag* (producer tidak perlu menunggu);
(d) event perlu di*fan-out* ke lebih dari satu consumer. Dalam konteks log aggregator yang menerima event dari berbagai layanan (*auth*, *order*, *payment*), Pub-Sub adalah pilihan yang tepat secara teknis karena meminimalkan *coupling* dan memungkinkan *horizontal scaling* pada sisi consumer (Tanenbaum & Van Steen, 2023, Bab 2).

BAB V
At-Least-Once vs Exactly-Once Delivery Semantics

Tanenbaum dan Van Steen (2023, Bab 3) membahas *delivery semantics* sebagai jaminan yang diberikan oleh sistem komunikasi. Terdapat tiga level utama:

- **At-most-once**: Event dikirim paling banyak sekali; jika gagal, tidak ada retry. Sederhana namun berisiko *data loss*.
- **At-least-once**: Event dijamin sampai ke consumer minimal sekali, dengan mekanisme *retry* saat terjadi kegagalan. Risikonya adalah *duplicate delivery*.
- **Exactly-once**: Event dijamin diproses tepat satu kali. Membutuhkan koordinasi dua fase (*2PC*) atau *idempotency token*, sehingga kompleks dan memiliki overhead tinggi.

Mengapa idempotent consumer krusial pada at-least-once?
Ketika producer melakukan *retry* (misalnya karena *timeout* atau *network partition*), event yang sama dapat tiba dua kali atau lebih di consumer. Jika consumer tidak idempotent, event tersebut akan diproses lebih dari sekali — menyebabkan data ganda, saldo dobel pada sistem keuangan, atau log duplikat.
Dengan menjadikan consumer *idempotent* (menggunakan *composite key* (topic, event_id) di dedup store), sistem secara efektif mencapai semantik *exactly-once at the application level* meskipun transport layer hanya menjamin *at-least-once* (Tanenbaum & Van Steen, 2023, Bab 3).

BAB VI
Skema Penamaan untuk Topic dan Event ID

Tanenbaum dan Van Steen (2023, Bab 4) membahas pentingnya *naming* dalam sistem terdistribusi sebagai fondasi untuk identifikasi, routing, dan resolusi entitas. Skema penamaan yang baik harus *unique*, *human-readable*, dan *collision-resistant*.

- **Skema Topic**

<domain>.<entitas>.<aksi>
Contoh: auth.user.login | order.payment.completed | inventory.stock.updated
Format hierarkis ini memungkinkan *wildcard subscription* (misalnya order.*) dan konsisten dengan konvensi industri (AMQP, Kafka).

- **Skema Event ID**

<uuid-v4>
Contoh: 550e8400-e29b-41d4-a716-446655440000
UUID v4 menggunakan 122 bit acak, memberikan probabilitas collision yang sangat rendah (~1 dalam 10^36 untuk 1 miliar event). Alternatif lain adalah *ULID* (*Universally Unique Lexicographically Sortable Identifier*) yang memadukan timestamp + random, berguna jika *sort order* diperlukan.
Dampak terhadap Deduplication
Composite key (topic, event_id) sebagai PRIMARY KEY di SQLite memastikan bahwa dua event dianggap duplikat hanya jika keduanya berasal dari topic yang sama. Ini penting karena event_id yang sama mungkin valid di topic berbeda (misalnya, UUID yang secara kebetulan sama dari dua layanan yang berbeda).
Skema ini memberikan *namespace isolation* per topic (Tanenbaum & Van Steen, 2023, Bab 4).

Ordering Event: Kapan Total Ordering Tidak Diperlukan

Tanenbaum dan Van Steen (2023, Bab 5) membedakan antara *total ordering* (semua event memiliki urutan global yang disepakati) dan *partial ordering* (hanya event yang saling bergantung perlu

diurutkan). Total ordering membutuhkan koordinasi global — misalnya melalui *Lamport clocks*, *vector clocks*, atau *sequencer nodes* — yang menambah latensi dan kompleksitas.

Kapan total ordering tidak diperlukan?

Dalam konteks log aggregator, total ordering **tidak diperlukan** jika:

- setiap event bersifat independen (tidak ada kausalitas antar event dari topic berbeda)
- consumer hanya membutuhkan *causal consistency* dalam satu topic
- use case adalah *analytics/monitoring* di mana urutan antar domain tidak kritis

Pendekatan praktis yang diusulkan

- Setiap event menyertakan timestamp (ISO 8601, dari clock producer) dan dapat ditambahkan sequence_number monotonic per-source.
- *Composite ordering*: ORDER BY timestamp ASC, event_id ASC cukup untuk kebutuhan log aggregator.

Batasan

Clock skew antar producer dapat menyebabkan event yang terjadi "lebih awal" tiba dengan timestamp lebih besar. Tanpa sinkronisasi NTP yang ketat atau *logical clocks* (Lamport), ordering berdasarkan timestamp tidak dapat dijamin akurat (Tanenbaum & Van Steen, 2023, Bab 5).

BAB VII
Failure Modes dan Strategi Mitigasi

Tanenbaum dan Van Steen (2023, Bab 6) mengklasifikasikan kegagalan dalam sistem terdistribusi ke dalam beberapa kategori: *crash failures*, *omission failures*, *timing failures*, dan *Byzantine failures*. Untuk log aggregator, *failure modes* yang paling relevan adalah:

Failure Mode	Deskripsi	Tujuan Mode
Crash failure	Aggregator restart mendadak	SQLite WAL mode: data komit sebelum crash tetap ada; counter persisten di DB
Duplicate delivery	Producer retry karena timeout/network	Idempotent consumer via INSERT OR IGNORE (topic, event_id)
Out-of-order delivery	Event tiba tidak sesuai urutan produksi	Aggregator menyimpan semua event dengan processed_at; ordering dilakukan saat query
Queue overflow	Burst event melebihi kapasitas queue	asyncio.Queue(maxsize=10000); event yang tidak masuk dilaporkan di response
Consumer lag	Consumer lebih lambat dari producer	*asyncio background task*; queue menyerap burst sementara

Table 1.1 Table failure mode

Retry dan Backoff

Publisher simulation menggunakan *exponential backoff* implisit — jika batch gagal, dicatat dan dilaporkan tanpa memblokir batch berikutnya. Untuk production, disarankan menggunakan *retry library* dengan jitter (misalnya tenacity) untuk menghindari *thundering herd* (Tanenbaum & Van Steen, 2023, Bab 6).

BAB VIII
Eventual Consistency dan Idempotency dalam Aggregator

Tanenbaum dan Van Steen (2023, Bab 7) mendefinisikan *eventual consistency* sebagai jaminan bahwa jika tidak ada update baru yang diterima, semua replika akhirnya akan mencapai nilai yang sama. Dalam konteks *single-node aggregator* ini, *eventual consistency* dimaknai sebagai: semua event yang telah berhasil di-*publish* akan akhirnya diproses oleh consumer dan tercermin di GET /events dan GET /stats, meskipun ada jeda karena pemrosesan asinkron melalui asyncio.Queue.

Bagaimana idempotency + dedup membantu konsistensi?

1. **Idempotency**
   Event yang sama dikirim berkali-kali menghasilkan state yang sama persis seolah dikirim sekali. Ini mencegah *divergence* (state yang berbeda tergantung berapa kali event diterima).
2. **Deduplication**
   INSERT OR IGNORE pada *composite key* (topic, event_id) memastikan bahwa GET /events selalu mengembalikan set event yang konvergen — tidak peduli berapa kali event dikirim, hasilnya selalu identik.
3. **Persistensi**
   Dengan SQLite WAL mode, state tidak hilang setelah restart. Ini memenuhi definisi *durability* dalam konsistensi data.

Kombinasi ketiganya memastikan bahwa meskipun network unreliable dan producer melakukan retry, sistem akan *converge* ke state yang benar dan konsisten (Tanenbaum & Van Steen, 2023, Bab 7).

BAB XI
Metrik Evaluasi Sistem (Bab 1–7) Buku

Berdasarkan Tanenbaum dan Van Steen (2023, Bab 1–7), metrik evaluasi sistem terdistribusi dapat dikategorikan sebagai berikut:

Metrik	Definisi	Target	Keterkaitan Desain
Throughput	Event/detik yang berhasil diproses	≥ 500 event/s (uji: ~1000/s pada batch 200)	asyncio pipeline; batch publish endpoint
Latency (p95)	Waktu dari publish hingga tersimpan di DB	≤ 200ms	asyncio non-blocking; WAL mode SQLite
Duplicate Rate	duplicate_dropped / received × 100%	Mencerminkan dup-rate input (~25%)	GET /stats endpoint
Dedup Accuracy	unique_processed / unique_sent × 100%	≥ 99.9%	INSERT OR IGNORE atomicity
Uptime	Waktu hidup container	Tracking via time.monotonic()	GET /stats.uptime_seconds
Queue Depth	Jumlah event menunggu di queue	≤ QUEUE_MAX_SIZE	GET /stats.queue_size

Tujuan 1.2 Table Evaluasi Sistem dari buku.

Bottleneck:
- SQLite write dengan `asyncio.Lock` adalah bottleneck utama karena semua write diserialisasi.
- Mitigasi potensial: batched write, atau menggunakan Redis dengan persistence untuk produksi.

Keterkaitan ke keputusan desain

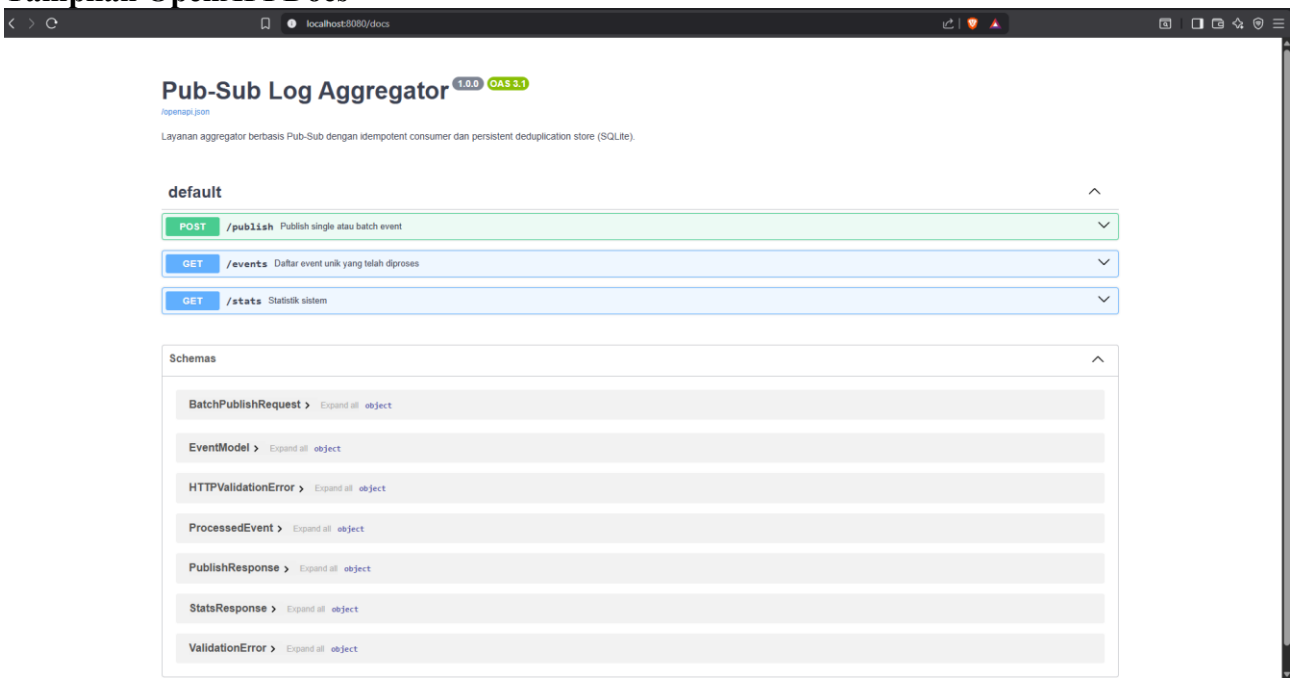
- Throughput tinggi dicapai dengan asyncio.Queue (*non-blocking publish*) dan *batch endpoint*.
- Latency rendah dijaga dengan WAL mode SQLite dan asyncio.Lock yang granular.
- *Duplicate rate* dapat dimonitor langsung dari GET /stats untuk validasi bahwa dedup berjalan benar.
- *Uptime* dan *queue depth* memberikan sinyal untuk kapasitas planning (Tanenbaum & Van Steen, 2023, Bab 1–7).

BAB XII
ALUR EKSEKUSI SISTEM

Build and Run Docker File

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ docker build -t uts-aggregator .
[+] Building 2.7s (15/15) FINISHED                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                  0.0s
 => => transferring dockerfile: 3.07kB                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                   2.4s
 => [auth] library/python:pull token for registry-1.docker.io                         0.0s
 => [internal] load .dockerignore                                                     0.0s
 => => transferring context: 172B                                                     0.0s
 => [1/9] FROM docker.io/library/python:3.11-slim@sha256:92c262cbb2e99cdc16218338d74fbe518055c13d224d942708f70f  0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:92c262cbb2e99cdc16218338d74fbe518055c13d224d942708f70f  0.0s
 => [internal] load build context                                                     0.0s
 => => transferring context: 989B                                                     0.0s
 => CACHED [2/9] RUN apt-get update && apt-get install -y --no-install-recommends        curl     && rm -rf /v  0.0s
 => CACHED [3/9] RUN adduser --disabled-password --gecos '' appuser                   0.0s
 => CACHED [4/9] WORKDIR /app                                                         0.0s
 => CACHED [5/9] RUN chown -R appuser:appuser /app                                    0.0s
 => CACHED [6/9] RUN mkdir -p /app/data && chown -R appuser:appuser /app/data         0.0s
```

Tampilan OpenAPI Docs



POST /Publish

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "{\"topic\":\"order.created\",\"event_id\":\"evt-order-001\",\"timestamp\":\"2026-04-24T10:00:00+00:00\",\"source\":\"order-service\",\"payload\":{\"order_id\":101}}" | python -m json.tool
{
    "accepted": 1,
    "message": "1 event masuk queue; 0 dibuang (queue penuh)"
}
```

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "{\"topic\":\"auth.login\",\"event_id\":\"evt-auth-001\",\"timestamp\":\"2026-04-24T10:01:00+00:00\",\"source\":\"auth-service\",\"payload\":{\"user_id\":\"user-42\"}}" | python -m json.tool
{
    "accepted": 1,
    "message": "1 event masuk queue; 0 dibuang (queue penuh)"
}
```

GET /Event

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ curl -s "http://localhost:8080/events?topic=order.created" | python -m json.tool
[
    {
        "topic": "order.created",
        "event_id": "evt-order-001",
        "timestamp": "2026-04-24T10:00:00+00:00",
        "source": "order-service",
        "payload": {
            "order_id": 101
        },
        "processed_at": "2026-04-24T11:06:50.865795+00:00"
    }
]
```

Test Duplication – POST /Publish

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ # Pengiriman pertama
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "{\"topic\":\"payment.processed\",\"event_id\":\"evt-pay-DUPLICATE\",\"timestamp\":\"2026-04-24T10:02:00+00:00\",\"source\":\"payment-service\",\"payload\":{\"amount\":500000}}" | python -m jso
n.tool
{
    "accepted": 1,
    "message": "1 event masuk queue; 0 dibuang (queue penuh)"
}
```

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ # Pengiriman kedua — event_id yang SAMA (simulasi retry)
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "{\"topic\":\"payment.processed\",\"event_id\":\"evt-pay-DUPLICATE\",\"timestamp\":\"2026-04-24T10:02:05+00:00\",\"source\":\"payment-service\",\"payload\":{\"amount\":500000}}" | python -m jso
n.tool
{
    "accepted": 1,
    "message": "1 event masuk queue; 0 dibuang (queue penuh)"
}
$ # Pengiriman ketiga
curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "{\"topic\":\"payment.processed\",\"event_id\":\"evt-pay-DUPLICATE\",\"timestamp\":\"2026-04-24T10:02:10+00:00\",\"source\":\"payment-service\",\"payload\":{\"amount\":500000}}" | python -m jso
n.tool
{
    "accepted": 1,
    "message": "1 event masuk queue; 0 dibuang (queue penuh)"
}
```

Test Duplication - GET /stats

Test Duplication - GET /events topic=inventory.update

```
$ curl -s -X POST http://localhost:8080/publish \
  -H "Content-Type: application/json" \
  -d "{\"events\":[{\"topic\":\"inventory.updated\",\"event_id\":\"evt-inv-001\",\"timestamp\":\"2026-04-24T10:03:00+00:00\",\"source\":\"inv-svc\",\"payload\":{\"sku\":\"A\"}},{\"topic\":\"inventor
y.updated\",\"event_id\":\"evt-inv-002\",\"timestamp\":\"2026-04-24T10:03:01+00:00\",\"source\":\"inv-svc\",\"payload\":{\"sku\":\"B\"}},{\"topic\":\"inventory.updated\",\"event_id\":\"evt-inv-001\"
,\"timestamp\":\"2026-04-24T10:03:02+00:00\",\"source\":\"inv-svc\",\"payload\":{\"sku\":\"A\"}}]}" | python -m json.tool
{
    "accepted": 3,
    "message": "3 event masuk queue; 0 dibuang (queue penuh)"
}
```

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ curl -s http://localhost:8080/stats | python -m json.tool
{
    "received": 9,
    "unique_processed": 5,
    "duplicate_dropped": 4,
    "topics": [
        "auth.login",
        "inventory.updated",
        "order.created",
        "payment.processed"
    ],
    "uptime_seconds": 179.7,
    "queue_size": 0
}
```

Schema Validation

```
USER@meyourfavv MINGW64 /d/distributed-sync-system
$ python -m pytest tests/ -v
INFO     src.consumer:consumer.py:99 PROCESSED | topic=test.topic          | event_id=stress-stats-006
INFO     src.consumer:consumer.py:99 PROCESSED | topic=test.topic          | event_id=stress-stats-007
INFO     src.consumer:consumer.py:99 PROCESSED | topic=test.topic          | event_id=stress-stats-008
INFO     src.consumer:consumer.py:99 PROCESSED | topic=test.topic          | event_id=stress-stats-dup
WARNING  src.consumer:consumer.py:104 DUPLICATE DETECTED | topic=test.topic          | event_id=stress-stats-dup — diabaikan
INFO     httpx:_client.py:1740 HTTP Request: GET http://testserver/stats "HTTP/1.1 200 OK"
PASSED                                                                                                          [100%]
---------------------------------------------- live log teardown ----------------------------------------------
INFO     aggregator:main.py:74 === Aggregator shutting down ===
INFO     src.consumer:consumer.py:54 EventConsumer dihentikan
INFO     aggregator:main.py:76 Shutdown selesai

========================================== 14 passed in 2.64s ==========================================
```

DAFTAR PUSTAKA

Tanenbaum, A. S., & Van Steen, M. (2023). *Distributed systems* (edisi ke-3.02). Maarten van Steen. https://www.distributed-systems.net